

C and C++ Memory Safety Foundations

Memory correctness is a contract about which object exists, who owns it, how it may be accessed, and when access ends. Learn these boundaries with small local programs, strong compiler warnings, sanitizers, and explicit cleanup reasoning. Keep C and C++ language rules distinct where they differ.

For: Engineers preparing for systems roles who need to reason about object lifetime, ownership, bounds, cleanup, and undefined behavior rather than rely on a program appearing to work once.

Suggested pace: 4 weeks with compiler-assisted labs | Basic C or C++ syntax required

What you will be able to do

- Identify stack, heap, static-storage, and object-lifetime boundaries.
- Document ownership transfer, borrowing, and cleanup for an API.
- Use warnings and sanitizers to investigate bounds and lifetime defects.
- Design error paths that release acquired resources exactly once.

Build the right mental model

Lifetime is more than an address

A pointer value can remain non-null after the referenced object's lifetime ends. Reusing the same address does not restore the old ownership contract. Define whether a function borrows data, copies it, or takes ownership, and how long the caller may retain a reference.

Bounds and representation matter

Track lengths and capacity separately. Account for terminators where required, check arithmetic before allocation, and avoid assuming serialized bytes are safely aligned native objects. Undefined behavior can invalidate reasoning even when a local run produces plausible output.

Use language tools appropriately

In modern C++, RAI and standard containers reduce manual cleanup, while smart pointers encode common ownership patterns. In C, explicit ownership conventions and structured cleanup are essential. Sanitizers find many bugs, but their absence does not prove correctness.

Study with evidence.

Keep exercises small and run them with warnings and suitable debug instrumentation. Explain the ownership graph before optimizing. Use synthetic buffers and local programs; the objective is defensive correctness, not exploiting a deployed process.

Week 1: Storage and lifetime

Trace local variables, dynamic allocations, returned values, and borrowed references. Write API comments that specify ownership and valid lifetime. Study copy, move, and destruction behavior if using C++.

Deliverable and check: A small ownership diagram and examples of safe return values versus invalid references to expired objects.

Week 2: Bounds and arithmetic

Practice buffers with explicit size and capacity, validated indexes, and checked allocation arithmetic. Test zero-length, maximum-size, and malformed input. Prefer standard containers where appropriate.

Deliverable and check: A parser or buffer utility with boundary tests and clear rejection of invalid sizes.

Week 3: Cleanup and errors

Exercise failures after each resource acquisition. In C, use a consistent cleanup path; in C++, use RAII objects with well-defined destruction. Ensure ownership is not duplicated during retries or partial initialization.

Deliverable and check: Tests showing every acquired resource is released once on success and each failure path.

Week 4: Concurrency and diagnosis

Study shared ownership, use-after-free risk across threads, and the distinction between atomic access and object lifetime. Use a debugger and sanitizers on local test cases, then explain the root cause and regression test.

Deliverable and check: A short diagnosis with the invalid lifetime transition and a minimal correction, not just a tool screenshot.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Design a safe resource-loading API

A function opens a file, allocates a buffer, parses metadata, and may fail at any stage. The caller needs a usable result on success and no leaked or dangling resources on failure.

A worked approach

1. List resources and owners at each stage: file handle, allocated storage, parsed object, and any borrowed input. Specify whether the final object owns the buffer or copies data into separate storage.
2. Choose one cleanup strategy. In C++, wrap resources in RAII types; in C, initialize handles to safe states and route failures through ordered cleanup. Publish the result to the caller only after all required invariants hold.
3. Inject a failure after each acquisition and test invalid metadata, empty input, and allocation failure where your harness supports it. Verify no result points to temporary storage and no resource is released twice.

Expected result

Success returns an object with an explicit ownership contract, while every failure leaves the caller without partial ownership. The design remains understandable even before instrumentation is run.

Verify the result

- Does every resource have exactly one responsible owner?
- Can the caller use the result after temporary parsing objects are destroyed?
- Do instrumentation and manual lifetime reasoning agree?

Extend the exercise

Add asynchronous parsing and define how the input buffer remains alive until completion or cancellation. An atomic pointer alone is not a lifetime-management strategy.

Make the learning stick.

Does setting a pointer to null fix use-after-free?

It helps only for that pointer variable and does not invalidate other aliases. Correctness requires controlling ownership and access across all references, not merely clearing one local copy after freeing.

Is shared_ptr always the safest choice?

It can represent shared lifetime, but adds cost and can create cycles or hide unclear ownership. Prefer unique ownership when possible, and use weak references deliberately for non-owning relationships that must observe expiration.

Do sanitizers prove the program is safe?

No. They detect classes of defects on executed paths and within their instrumentation model. Combine them with tests, reviews, ownership contracts, and careful reasoning about unexecuted paths and concurrency.

Avoid these traps

- Equating non-null pointers with live valid objects.
- Ignoring integer overflow when computing allocation sizes.
- Fixing a symptom without defining who owns the resource.

Ready for the next step?

- Ownership and borrowing are explicit in public interfaces.
- Boundary and partial-initialization cases have tests.
- Cleanup behavior is correct on every exit path.
- A memory defect can be explained as a broken invariant.

Reliable next reads

- [C++ Core Guidelines](#)
- [Clang AddressSanitizer](#)
- [SEI CERT C Coding Standard](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.